

CPSC 3720 Assignment #3

Due Date: November 7, 2025

By: Adrian Todd

Theory Questions:

1. Give five circumstances where you might recommend against software reuse.
 - a. I would recommend against software reuse if the software component is unmaintained, for example a library that has not been updated in 10 years. If a component like this is used there could be security vulnerabilities, and you could get stuck with having to fix bugs or known security issues.
 - b. I would recommend against software reuse if there is not good clear documentation otherwise you may run into issues with components performing unexpectedly or integration issues.
 - c. I would recommend against software reuse when you are working in a resource constrained environment like an embedded system. Although there may be a general-purpose library component that does what you need it may also do a thousand other tasks that you don't need so it may take up significant memory space that is not available or could be better used.
 - d. I would recommend against software reuse if the component does less than 50 percent of what you want it to do, in this case the cost, time, and complexity of modifying the existing component may end up exceeding the cost of building a custom component.
 - e. I would recommend against software reuse when it has restricted licensing that does not match up with your requirements. For example, software that has a General-Purpose License must also be reused as open-source software and not integrated into a closed source software product.

2. Why do many large organizations adopt ERP systems for their information infrastructure? What reuse-related challenges might arise during ERP deployment?

Large organizations often adopt ERP systems to solve the problem of fragmentation. ERP systems can take several or all the separate software systems that a company uses such as a finance system, a human resources system, a warehouse inventory management system, etc... and the ERP systems purpose is to replace all these separate systems with one single integrated system. ERP systems provide several key benefits to large organizations. They can provide a single source of truth where everyone in the company is working from the same real-time data which in turn can

improve reporting and analytics. They provide consolidation and cost savings by reducing IT complexity and maintenance costs.

Some reuse related challenges that large organizations may run into is that the ERP may not be able to do everything the company needs and so custom code to change the ERP so that it can accommodate additional tasks needs to be written which breaks the reuse model. Another reuse related challenge is integration with legacy systems that cannot feasibly be replaced, this can again lead to custom code being written to try and get the systems to talk to one another.

Project: Reusable Data Visualization Dashboard Framework for Alberta Energy Data

Abstract

Analyzing Alberta's electricity demand is complex due to high volatility and external factors like weather. This project focuses on developing a data visualization dashboard framework from reusable software components. The framework integrates Alberta Electric System Operator (AESO) data from a local CSV with historical weather data from the remote Open-Meteo API, allowing users to explore correlations between energy demand, price, and temperature. My goal is to demonstrate software reuse by integrating flexible and maintainable components to develop a dashboard that dynamically adjusts all visualizations according to the user's needs.

Objectives

- Develop a data dashboard from reusable software components.
- Demonstrate a modular and maintainable design.
- Analyze multiple datasets (local CSV and remote API).
- Implement useful, multi-faceted data visualizations.
- Allow users to compare datasets and discover correlations.

Software Requirements

Functional Requirements (FRs)

- FR1: The system must load and process data from a local CSV file (AESO) and a remote JSON-based API (Open-Meteo).
- FR2: The user must be able to select a custom date range for analysis.
- FR3: The user must be able to adjust the parameters of the analysis

- FR4: The data and visualizations must be reprocessed and update in real-time in response to user interaction.
- FR5: The system must display multiple visualization types

Non-functional Requirements (NFRs)

- NFR1 (Performance): All data loading and API calls must be cached. Subsequent user interactions (adjusting sliders, changing dates) must result in a visualization update in under 5 seconds.
- NFR2 (Usability): The dashboard must be intuitive for a non-technical user with no prior training. All controls must be clearly labeled.
- NFR3 (Maintainability): The system's architecture must separate the User Interface (UI) logic from the data processing and visualization logic.
- NFR4 (Portability): The application must be cross-platform, runnable on any system that has Python and a modern web browser.

Methodology

The project followed a component-based software engineering approach:

1. Reusable Software Component Review: Identified Streamlit as the dashboard framework, Pandas for data manipulation, Seaborn for visualization, and Requests for API communication.
2. System Design and Architecture: Designed a two-file modular architecture (View/Controller) to ensure maintainability.
3. Data Loader and Visualization Component Implementation: Encapsulated all data loading and plotting logic into a single reusable module (`dashboard_helpers.py`).
4. Data Dashboard Implementation and Integration: Developed the main UI file (`dashboard.py`) which imports and integrates the components from the helper module.

System Design

The system is designed using a Separation of Concerns principle, which is fundamental to reusable and maintainable software. The architecture is split into two primary modules:

1. `dashboard.py` (The View/UI Layer):
 - This file's sole responsibility is to render the web application's User Interface.

- It imports streamlit to create widgets like `st.title`, `st.sidebar.slider`, and `st.sidebar.date_input`.
- It contains no data processing logic. It simply calls functions from the helper module.
- It receives data and Figure objects from the helper module and displays them using `st.pyplot` and `st.dataframe`.

2. `dashboard_helpers.py` (The Model/Controller Layer):

- This file is a reusable module that contains all the project's "business logic" and can be reused by any other script.
- Data Components: Contains functions like `load_aeso_data()`, `load_daily_weather_data()`, and `load_hourly_weather_data()`. These components handle loading, cleaning, and caching data from both the CSV and the remote API.
- Visualization Components: Contains functions like `create_aeso_plot()` and `create_demand_temp_plot()`. These components take clean data as input and return a complete `matplotlib.Figure` object, ready for display.

This design ensures that to change the UI, only `dashboard.py` needs to be changed. To change the logic only `dashboard_helpers.py` needs to be changed.

Reused Components:

- Framework: Streamlit
- Data Manipulation: Pandas
- Data Visualization: Matplotlib, Seaborn
- API Communication: Requests
- External Data Service: Open-Meteo Historical Weather API

Datasets

- AESO Data (Local CSV): `Hourly_Metered_Volumes_and_Pool_Price_and_AIL_2020-Jul2025.csv`. This dataset provides hourly Alberta Internal Load (demand) and Pool Price.
- Historical Weather Data (Remote API): Sourced from the Open-Meteo Historical Weather API. This project requests hourly and daily mean temperature for Calgary, AB.

Results

The project successfully produced a flexible and maintainable data dashboard framework. The dashboard effectively integrates and displays data from two different sources, allowing for complex analysis.

- **Component Reuse:** The `dashboard_helpers.py` module is 100% reusable. The application's functionality is built almost entirely from external reused components (Streamlit, Pandas, Matplotlib, Seaborn, Requests) and internal reusable components (`load_data`, `create_plot`).
- **Key Findings:** The dashboard visualizations successfully met the project objectives.
 1. The 3-panel time-series plot clearly shows a visual correlation between temperature, demand, and price
 2. The daily correlation scatter plot reveals a strong, non-linear relationship between temperature and demand, where demand peaks at both very cold ($< -10^{\circ}\text{C}$) and very hot ($> 20^{\circ}\text{C}$) temperatures.
 3. The `st.metric` component provided a quantitative correlation value, confirming the visual trend.

Conclusions

- The Separation of Concerns 2-file architecture was extremely effective. It made the code easy to debug and will make it simple to expand for the final project.
- Reusing Streamlit's `@st.cache_data` component was critical. Without it, the API would be called on every widget click, making the app slow and unusable.
- Reusing the Open-Meteo API as an external data component was simple and added immense analytical value, allowing the project to move beyond simple data plotting into true correlation analysis.

What Could Be Improved:

- In the future I think that the data dashboard could be improved by incorporating AESO data that shows where the electricity is generated from like solar, hydro, coal power plants etc.
- I would also like to add in additional functionality to the data visualizations to make them more interactive. I was learning about Plotly and think that I could use that library to make the dashboard more interactive like adding the ability to zoom into sections of the visualizations or hover over certain sections to get specific data.

References

- Sommerville, I. (2020). *Software Engineering* (10th Ed.). Pearson.
- Gemini AI (Used for suggestions on what components might be useful to reuse for this project and why they would be beneficial)
- Alberta Electric System Operator (AESO). (2025). *Current and Historical Market Data*. Retrieved from <https://www.aeso.ca/market/market-and-system-reporting/>
- Open-Meteo (2025). *Open-Meteo Historical Weather API*. Retrieved from <https://open-meteo.com/en/docs/historical-weather-api>
- Python libraries:
 - Streamlit
 - Pandas
 - Matplotlib
 - Seaborn
 - Requests